

From Behaviors to Contextual Refinement

Program Verification Workshop 2026

Day 1 · CRIS fundamentals and guided proofs

What can this program do?

```
Check Beh.of_itree.  
(* itree coreE Any.t -> Tr.t -> Prop *)
```

Beh.of_itree program trace says that trace is one behavior of program .

| Read it as trace membership.

Return produces a terminating trace

```
Definition return_42 : itree coreE Any.t :=  
  Ret (42%Z)↑.
```

```
Check (Tr.done (42%Z)↑).
```

```
Check (Beh.of_itree return_42 (Tr.done (42%Z)↑)).
```

Ret 42 $\longrightarrow$ done 42

Tau preserves the observation

```
Definition silent_return_42 : itree coreE Any.t :=  
  tau;; return_42.
```

```
tau; Ret 42  —————> done 42  
Ret 42      —————> done 42
```

Tau is an internal computation step.

Choose gives several behaviors

```
Definition choose_bit : itree coreE Any.t :=  
  'b : bool <- trigger (Choose bool);;  
  Ret ((if b then 1%Z else 0%Z)↑).
```

```
choose_bit → done 0  
choose_bit → done 1
```

One program supports both membership propositions.

I/O extends the trace

```
Definition echo_once : itree coreE Any.t :=  
  'answer : Z <- trigger (@IO Z Z "echo" 7%Z);;  
  Ret answer↑.
```

```
request echo(7)  
  ┆ response 9 → interact echo(7 ↦ 9); done 9  
  ┆ unanswered → hang echo(7)
```

Trace shapes

```
Print Tr.t.
```

```
done(ret)      abort      spin  
hang(request)  interact(io, rest)
```

- `spin` : infinitely many internal steps
- `interact` : one observable I/O followed by the remaining trace
- an infinite I/O behavior: an infinite chain of `interact`

Where does the closed interaction tree come from?

```
? : itree coreE Any.t
```

|



```
Beh.of_itree
```

Open `day1/lectures/ModuleIntro.v`.

day1/lectures/ModuleIntro.v

A module packages code and state

```
Check SMod.fnsems.  
Check SMod.initial_st.
```

```
module = named function bodies + private initial state + scope
```

Function bodies have type `itree crise ...`.

A header names a typed boundary

```
Module ConstHdr.  
  Definition get :=  
    fnsig (fn "get") (fntyp () Z).  
End ConstHdr.
```

```
get : unit → Z
```

Callers and implementations share this signature.

The service implements the boundary

```
Definition get : () -> itree crisE Z :=  
  fun _ => Ret 42%Z.
```

```
Program Definition smod : SMod.t := {|  
  SMod.fnsems := fnsems;  
  SMod.initial_st := ∅;  
|}.
```

The first service uses empty local state.

The client performs a call

```
Definition main : Any.t -> itree crisE Any.t :=  
  fun _ =>  
    n <- ccallU ConstHdr.get tt;;  
    Ret n↑.
```

MainModule.entry — Call get() → ConstModule.get

Link, compile, observe

Definition `modules : Mod.t :=`
 `MainModule.t ★ ConstModule.t.`

Definition `program : itree coreE Any.t :=`
 `LMod.compile (Mod.to_lmod modules ε) tt↑.`

Check `(Beh.of_itree program).`

Private state will matter soon

```
Check cgetU.  
Check cput.
```

```
SGet key      reads the current module-local value  
SPut key value updates the module-local value
```

Compilation interprets both operations while producing the closed tree.

When may another service replace this one?

```
MainModule ★ SourceService
```

```
MainModule ★ TargetService
```

The client was chosen today. A reusable replacement theorem must support clients chosen later.

Open `day1/lectures/RefinementIntro.v` .

Target behaviors are allowed by the source

$$\text{Beh}(\text{Target}) \subseteq \text{Beh}(\text{Source})$$

- **Source** : the reference program or specification
- **Target** : the implementation we plan to run

The target introduces no behavior outside the source set.

Closed-program refinement

```
Print refines_lmod.
```

Read its body as:

```
Beh.of_itree (compile Target)  
  ⊆  
Beh.of_itree (compile Source)
```

A context supplies the rest of the program

```
Print ctx_refines.
```

```
for every Ctx,  
  refines (Target ★ Ctx) (Source ★ Ctx)
```

The same context is linked to both sides.

Keep the argument order visible

behavior inclusion

$\text{Beh}(\text{Target}) \subseteq \text{Beh}(\text{Source})$

contextual refinement

$\text{ctx_refines } \text{Target } \text{Source}$

simulation

$\text{ISim.t } \dots \text{Source Target } \dots$

The proof challenge

$$\begin{array}{cccc} \text{Ctx}_1 & \text{Ctx}_2 & \text{Ctx}_3 & \dots \\ \backslash & | & / & \\ \text{Source} & \sim & \text{Target} & \end{array}$$

`ctx_refines` quantifies over every context.

We want one local proof that composes with all of them.

Simulation discharges the contextual claim

per-function simulations

|



ISim.t open Source Target IC Ist

|

| main_adequacy



IC $\vdash$ ctx_refines Target Source

What does an actual simulation proof look like?

Start with empty local state.

```
Source: return (1 + 2)
```

```
Target: return 3
```

Open `day1/exercises/Optimizations.v`.

One interface, two bodies

```
(* Source *)  
fun _ => Ret ((1 + 2)%Z)  
  
(* Target *)  
fun _ => Ret 3%Z
```

The target has performed constant folding.

The first state relation is True

```
Definition Ist : ist_type  $\Sigma$  :=  
  fun _ _ => True%I.
```

Both modules start with empty local state.

Every pair of local states satisfies this first relation.

A function simulation

```
Lemma simF_fold_constants :  
  ISim.sim_fun open Source Target Ist  
    (fid PureOptHdr.fold_constants).
```

```
one exported function  
  source body ~ target body
```

Process one proof step at a time

```
cStartTypedFunSim u.  
unfold PureOptSource.fold_constants,  
       PureOptTarget.fold_constants.
```

Now inspect the goal:

```
source: Ret (1 + 2)  
target: Ret 3  
post:   restore Ist
```

STOP 1 · Constant folding

Complete `simF_fold_constants` .

```
(* match the returns *)  
(* restore Ist *)
```

Replace `Admitted` with your proof and `Qed` .

Reference solution: `day1/answers/Optimizations.v`

Assemble the module proof

```
Lemma sim : ISim.t open Source Target emp%I Ist.
```

```
Proof.
```

```
  cStartModSim.
```

```
  - iIntros "_". done.
```

```
  - apply simF_fold_constants.
```

```
Qed.
```

```
Lemma ctxr :  $\vdash$  ctx_refines Target Source.
```

```
Proof. eapply main_adequacy, sim. Qed.
```

Store-to-load forwarding

```
(* Source *)  
cput cell x;;;  
y <- cgetU cell;;  
Ret y
```

```
(* Target *)  
cput cell x;;;  
Ret x
```

Both stores are internal module steps.

The relation carries one integer

```
Definition Ist : ist_type  $\Sigma$  :=  
  fun st_src st_tgt =>  
    ( $\exists$  x : Z,  
       $\ulcorner$ st_src = {[source_cell # x $\uparrow$ ]} /\  
        st_tgt = {[target_cell # x $\uparrow$ ]} $\urcorner$ )%I.
```

```
source cell: x ~ target cell: x
```

Open, step, restore

```
initial relation      (old, old)
source store         (x,   old)
target store         (x,   x)
new relation witness  x
```

```
iDestruct "IST" as (old) "%".
cStepsS. cStepsT.
```

STOP 2 · Restore the cell relation

Complete `simF_store_load` .

```
(* match the final returns *)  
(* choose x as the new relation witness *)
```

Replace `Admitted` with your proof and `Qed` .

Reference solution: `day1/answers/Optimizations.v`

What if the states use different representations?

equal integer cells

↓

abstract map ~ sorted list

Open `day1/exercises/KVSortedList.v` .

day1/exercises/KVSortedList.v

One API, two private representations

```
get : Z → option Z  
put : Z × Z → unit
```

```
Source state : Z → option Z  
Target state : list (Z × Z)
```

Clients see the API. The module owns the representation.

Source lookup is immediate

```
Definition get : Z -> itree crisE (option Z) :=  
  fun k =>  
    'm : amap <- cgetU v_map;;  
    Ret (m k).
```

```
Definition put : Z * Z -> itree crisE () :=  
  ...  
  cput v_map (map_put m k v);;;  
  Ret tt.
```

Target state is a sorted mathematical list

Definition `entries := list (Z * Z).`

Fixpoint `list_get (q : Z) (xs : entries) : option Z := ...`

Fixpoint `list_put (k v : Z) (xs : entries) : entries := ...`

- keys are strictly increasing
- equal keys are overwritten
- lookup stops once the head key exceeds the query

The representation relation

```
Definition state_rel (m : amap) (xs : entries) : Prop :=  
  sorted_keys xs /\  
  forall k, m k = list_get k xs.
```

sortedness	supports ordered insertion and early lookup exit
lookup agreement	connects every observable get result

Provided algebra for updates

```
sorted_keys_put :  
  sorted_keys xs ->  
  sorted_keys (list_put k v xs)  
  
list_get_put :  
  list_get q (list_put k v xs) =  
  map_put (fun q => list_get q xs) k v q  
  
state_rel_put :  
  state_rel m xs ->  
  state_rel (map_put m k v) (list_put k v xs)
```

STOP 3-4 · Establish the pure relation

1. Prove `state_rel_empty` .
2. Prove `state_rel_put` using the provided lemmas.

```
empty_map ~ []
```

```
map_put m k v ~ list_put k v xs
```

Reference solution: `day1/answers/KVSortedList.v`

Put repeats the restoration pattern

```
pre-state      m      ~  xs
source update  map_put m k v
target update  list_put k v xs
post-state     related by state_rel_put
```

```
pose proof (state_rel_put m xs k v Hrel) as Hupdated.
cFinishKVReturn Hupdated.
```


STOP 5 · KV put simulation

Complete `simF_put`.

```
pose proof (state_rel_put m xs k v Hrel) as Hupdated.  
cFinishKVReturn Hupdated.
```

`Hupdated` is the mathematical step; the second tactic handles CRIS bookkeeping.

Reference solution: `day1/answers/KVSortedList.v`

Lookup carries a cursor

```
Definition get : Z -> itree crisE (option Z) :=
  fun q =>
    'xs : entries <- cgetU v_entries;;
    ITree.iter
      (fun rest : entries =>
        match rest with
        | [] => Ret (inr None)
        | (k, v) :: tl =>
          match Z.compare q k with
          | Lt => Ret (inr None)
          | Eq => Ret (inr (Some v))
          | Gt => Ret (inl tl)
        end
      end)
    xs.
```

One skipped head produces Tau

```
match Z.compare q k with  
| Lt => Ret (inr None)  
| Eq => Ret (inr (Some v))  
| Gt => Ret (inl tl)  
end
```

`ITree.iter` interprets `inl tl` as continuation with the tail.

Gt branch: one silent step, then search `tl`

The proof follows the cursor

```
pose proof Hrel as Hstate.  
destruct Hrel as [_ Hlookup]. rewrite Hlookup.  
generalize xs at 1 3. iIntros (cursor).  
iInduction cursor as [| [k' v'] tl] "IH".
```

Cursor case	Proof action
[]	unfold iterator, return None
Lt	return None
Eq	return Some v'
Gt	take target Tau , apply IH

STOP 6 · Iterator induction

Complete `simF_get`.

```
iInduction cursor as [|k' v'] tl] "IH".
```

Use `cFinishKVReturn Hstate` in the terminating cases.

Use `cStepT; iApply "IH"` in the `Gt` case.

Reference solution: `day1/answers/KVSortedList.v`

Assemble and return to behaviors

Lemma `sim : ISim.t open Source Target emp%I Ist.`

Lemma `ctxr : ⊢ ctx_refines Target Source.`

`state_rel_empty`

|

`simF_put + simF_get`

|

▼

`module simulation`

|

`main_adequacy`

▼

`ctx_refines SortedListTarget KVSource`

Back to the first question

sorted-list target + any compatible client
| link and compile
▼
target behavior
⊆
source behavior

Every target behavior is permitted by the abstract key-value source.